



Security Assessment Final Report



Safenet Guard

August 2026

Prepared for Safe

Table of Contents

Project Summary	3
Project Scope	3
Project Overview	3
Protocol Overview	3
Assessment Methodology	4
Threat and Security Overview	5
Assets	5
Actors	6
Roles	7
Assumptions	8
Attack Surface	9
Safe authorization and transaction binding	9
Attestations, oracles, and relayers	10
Epoch rollover and historical keys	11
Announcement escape hatch	12
Safe integration and execution	13
Deployment, encoding, and observability	14
Findings Summary	15
Severity Matrix	15
Detailed Findings	16
Critical Severity Issues	17
C-01: Auto-allowed calls can drain Safe assets through unattested gas refunds	17
Low Severity Issues	20
L-01: Indefinite attestations allow transactions to outlive the policy that approved them	20
L-02: Announcement state becomes permissionless when the guard is configured as the Safe's fallback handler	23
Informational Severity Issues	25
I-01: Historical FROST keys remain permanently authoritative on target chains	25
I-02: Guard entry points accept calls from accounts that do not use the guard	27
Disclaimer	28
About Certora	28

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Safenet Guard	https://github.com/safe-research/safenet	dbfd8fd	Solidity

Project Overview

This document describes the security review of **Safenet Guard**. The work was undertaken from **August 12th, 2026** to **August 18th, 2026**.

The following contract list is included in our scope:

`contracts/src/guard/SafenetGuard.sol`

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the Findings Summary page.

Protocol Overview

Safenet adds a second authorization layer to owner-signed Safe transactions. **SafenetGuard** is installed as a Safe transaction guard and checks each **execTransaction** before the Safe executes it. A transaction is permitted when it carries a valid Safenet FROST threshold-signature attestation, when it matches a matured time-delayed announcement, or when it is an auto-allowed call used to manage that announcement mechanism.

The guard runs on the Safe's target chain. It does not call Gnosis Chain or execute transactions through **FROSTCoordinator**. Instead, it keeps a local set of trusted Safenet epoch keys, reconstructs the attested EIP-712 message, verifies the FROST signature, and returns control to the Safe, which performs the actual call or delegatecall.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

The primary path is an owner-signed Safe `execTransaction` checked by `SafenetGuard`, with `AttestationTrailer`, `ConsensusMessages`, `EpochRollover`, `SafeTransaction`, `SignatureExtension`, and `TransactionAnnouncement` providing the relevant hashing, decoding, epoch, and escape-hatch logic. `Consensus`, `FROSTCoordinator`, validators, sentinels, and oracle contracts are modeled as external context rather than target-chain execution components.

Verdicts describe the implementation state at the time of this review:

- **Impossible** means an enforced invariant or access restriction prevents the threat.
- **Mitigated** means the threat is possible in principle but sufficiently constrained.
- **By design** means the behavior is intentional and its tradeoff is accepted.
- **Audit finding** means a concrete issue identified during this review and tracked in the current audit report. “Audit finding” does not imply prior unawareness or lack of acknowledgment by the project team.

Assets

- **Safe-controlled assets and authority** — a permitted Safe transaction may transfer native currency or tokens, call arbitrary contracts, delegatecall into Safe storage, or modify owners, modules, fallback handling, and the guard itself.
- **Two-layer authorization boundary** — immediate execution is intended to require both the Safe owner threshold and a Safenet FROST attestation. The announcement escape hatch replaces the Safenet approval with a fixed delay and bounded execution window.
- **Safe transaction authorization** — owner signatures, the Safe address, target-chain ID, transaction nonce, call parameters, operation, and refund parameters determine the exact Safe transaction being executed.
- **FROST signing authority** — validator secret shares and their aggregate group keys control the Safenet attestation layer. A threshold can produce valid transaction attestations and epoch-rollover certificates.

- **Trusted epoch forest** — the guard's local set of (`groupKey`, `epoch`) pairs determines which current or historical FROST keys may authorize transactions and additional rollover branches.
- **Oracle decision provenance** — the signed transaction proposal binds the oracle address and `keccak256(oracleData)`. Validators decide off-chain which oracle results they are willing to attest.
- **Announcement state** — the announcement hash, activation time, expiry time, and single-use state determine when an exact transaction may execute without a Safenet attestation.
- **Replay and freshness state** — the Safe nonce prevents replay of the same attested Safe transaction. Attestations otherwise have no deadline or on-chain revocation state.
- **Immutable deployment configuration** — the Consensus chain ID and address determine the EIP-712 domain, while the initial epoch key and fixed announcement delay and window determine the guard's trust root and recovery timing.

Actors

- **Individual Safe owner — semi-trusted:** can sign transactions but is bounded by the Safe threshold.
- **Safe owner quorum — time-delayed authority:** can satisfy Safe signature checks, create or cancel announcements, and ultimately execute arbitrary Safe actions after the announcement delay. It should not be able to execute arbitrary actions immediately without Safenet approval.
- **Individual validator — semi-trusted:** holds one FROST share and participates in signing and rollover ceremonies but cannot produce a threshold signature alone.
- **FROST validator threshold — authorization-critical:** can attest arbitrary transaction proposals and sign epoch rollovers. It cannot execute from a Safe without the required Safe owner signatures.
- **Oracle and sentinel policy — policy-trusted:** determine whether validators consider a proposed transaction acceptable. The target-chain guard authenticates the oracle context but does not evaluate it.

- **Consensus and FROSTCoordinator — system context:** coordinate proposals, oracle results, signing ceremonies, and epoch transitions on Gnosis Chain. They do not execute the attested Safe transaction on the target chain.
- **Relayer, transaction submitter, and block builder — untrusted:** can observe, copy, order, delay, censor, and submit signed Safe transactions, attestations, announcements, and rollover certificates. They do not receive authorization merely by submitting them.
- **Epoch updater — untrusted:** anyone may call `updateEpoch`; the parent FROST signature is the authorization.
- **Execution target — untrusted for calls, operation-trusted for delegatecalls:** a call target receives only the authorized call, while a delegatecall target executes with the Safe's storage and authority for that transaction.
- **Safe proxy and singleton — system-trusted:** hold the assets and owner state, verify owner signatures, manage the nonce, invoke the guard hooks, execute the transaction, and process gas refunds.
- **Enabled Safe module — fully trusted or explicit bypass:** module transactions do not invoke Safe transaction-guard hooks and can move Safe assets without a Safenet attestation or announcement.
- **Guard deployer and Safe integrator — trusted at deployment:** select the Consensus domain, genesis epoch key, timing parameters, Safe implementation, guard slot, modules, and fallback handler.

Roles

- **Immediate authorization requires two independent layers:** the Safe owner threshold authorizes the Safe transaction, and the FROST validator threshold authorizes the corresponding Safenet transaction proposal.
- **The owner quorum has eventual authority:** it can use the announcement path to execute an arbitrary exact transaction after the configured delay, including transactions that alter or remove the guard.
- **The validator threshold is the second-factor authority:** it can approve immediate execution but cannot bypass the Safe's owner-signature requirement.

- **The oracle is an off-chain policy input:** its address and data hash are authenticated by the FROST signature, but acceptable oracle implementations and results are validator policy rather than target-chain guard policy.
- **The Safe implementation is execution-critical:** the guard relies on Safe's signature verification, nonce behavior, hook ordering, failure handling, and gas-refund semantics.
- **Deployment configuration is a trust root:** the guard has no administrator, upgrade path, domain update, epoch reset, or timing update after deployment.
- **Relayers and callers are untrusted:** authorization comes from Safe signatures, FROST signatures, and announcement state, not from the identity of the transaction submitter.

Assumptions

- The guarded account uses a compatible Safe implementation that invokes `checkTransaction` before every owner-signed `execTransaction` and `checkAfterExecution` afterward.
- The guard is correctly installed in the Safe's guard slot. It is not also configured as the Safe's fallback handler.
- The Safe has no enabled modules unless each module is explicitly trusted as a complete bypass of the transaction guard.
- At least one of the two immediate authorization layers remains honest. A compromised Safe owner threshold should remain constrained by honest Safenet validators, while a compromised FROST threshold should remain constrained by honest Safe owners.
- Validators attest only transaction proposals whose oracle and oracle result they honor, and they verify the exact Safe transaction hash and oracle context before producing shares.
- Historical FROST secret shares are securely destroyed after rotation and cannot be recovered from backups, snapshots, retired machines, logs, or incident-response images.
- The configured Consensus chain ID, Consensus address, initial epoch, initial group key, announcement delay, and announcement window are correct.

- The EIP-712 schemas and domain separators used by Consensus, validators, and the target-chain guard are identical.
- Valid rollover certificates are delivered to each target-chain guard before transactions begin using the corresponding new group key. Temporary relay failure is treated as an availability issue.
- Wallet software presents all Safe transaction fields accurately, including gas-refund fields, and correctly constructs the signature-extension trailer and announcement parameters.
- Safe owners do not release a fully signed announcement-path transaction until they are prepared for any untrusted party who obtains it to submit it.
- Target-chain timestamps provide sufficient accuracy for the configured announcement delay and execution window.
- The secp256k1 and FROST verification logic is cryptographically sound, and fewer than the threshold of live validator shares are compromised.

Attack Surface

Safe authorization and transaction binding

- **A compromised owner quorum executes an arbitrary transaction immediately — Verdict: Mitigated** — ordinary owner-signed transactions still require either a valid FROST attestation or a matching matured announcement. The auto-allowed refund exception is covered separately as an audit finding.
- **A FROST threshold executes without Safe owner signatures — Verdict: Impossible** — the guard is reached from Safe's owner-signed `execTransaction` path, and Safe independently enforces its owner threshold.
- **Compromise of both authorization thresholds permits arbitrary immediate execution — Verdict: By design** — the owner threshold can authorize the Safe transaction and the FROST threshold can attest its exact hash. No third authorization layer remains.
- **A transaction field is changed after attestation — Verdict: Mitigated** — the attestation covers the nonce-bound Safe transaction hash, including target, value, calldata, operation, gas fields, gas token, refund receiver, Safe address, target-chain ID, and nonce.

- **Cross-Safe, cross-chain, or cross-nonce attestation replay — Verdict: Mitigated** — the Safe transaction hash binds the Safe, target-chain ID, and nonce. The outer Consensus EIP-712 domain additionally binds the Gnosis Chain ID and Consensus contract address.
- **A malformed recognized trailer silently falls through to the announcement path — Verdict: Impossible** — a recognized signature-extension type hash commits processing to the attestation path. Malformed envelopes, malformed payloads, unknown epoch keys, and invalid FROST signatures revert.
- **An auto-allowed announcement-management transaction transfers assets through Safe's refund mechanism — Verdict: Audit finding (C-01)** — `_isAutoAllowed` checks the inner call's target, value, operation, and selector but does not constrain `baseGas`, `gasPrice`, `gasToken`, or `refundReceiver`. A compromised owner quorum can use Safe's independent gas refund to drain assets without an attestation or matured announcement.
- **A valid attestation remains executable after the approving policy changes — Verdict: Audit finding (L-01)** — `TransactionProposal` has no expiry or policy version, and the guard has no revocation registry. The Safe nonce prevents replay but does not provide freshness.

Attestations, oracles, and relayers

- **A relay substitutes a different oracle or oracle result — Verdict: Mitigated** — the FROST-signed proposal binds both the oracle address and `keccak256(oracleData)`. Any substitution invalidates the signature.
- **The guard accepts an oracle that target-chain code does not recognize — Verdict: By design** — the guard pins neither the oracle nor its data format. Validators are expected to attest only results from oracle implementations they honor.
- **A permissionless caller proposes or submits a transaction before the intended proposer — Verdict: Mitigated** — proposal and relay permissions do not grant signing authority. The final guard authorization depends on the exact threshold-signed proposal, not the identity or ordering of the proposer.
- **FROSTCoordinator or Consensus executes the target-chain Safe transaction — Verdict: Impossible** — these components produce and coordinate authorization data. A relay submits the owner-signed transaction and attestation to the Safe on the target chain; the Safe executes after the guard returns successfully.

- **A relay strips a valid attestation trailer and spends a matching matured announcement — Verdict: By design** — owner signatures do not cover the appended signature extension. A relay holding both authorization paths may choose the announcement path; the exact transaction still requires owner signatures and a matured announcement.
- **A relay censors, delays, copies, or front-runs an authorized transaction — Verdict: By design** — signed Safe transactions and attached attestations are public bearer authorizations. The Safe nonce prevents replay of an attested transaction but not censorship or adversarial ordering.

Epoch rollover and historical keys

- **An arbitrary caller installs an unauthorized epoch key — Verdict: Mitigated** — `updateEpoch` requires a known parent pair, a strictly greater epoch, a nonzero successor key, and a valid FROST signature over the rollover message. Re-submitting a known pair is idempotent.
- **A validly signed branch is accepted without proving that Consensus activated it — Verdict: By design** — the guard maintains a forest rather than a canonical active epoch. Every valid parent-signed branch is accepted, multiple keys per epoch are permitted, and `rolloverBlock` is authenticated but not compared with the target chain's unrelated block number.
- **A historical group key attests future transactions or signs another rollover branch — Verdict: By design** — every recorded `(groupKey, epoch)` pair remains trusted forever. The stated rationale is that old secret shares are destroyed after rotation.
- **Recovered historical shares restore permanent Safenet signing authority — Verdict: Audit finding (I-01)** — a threshold of historical shares can create valid attestations or introduce another key without a current Coordinator ceremony or proof of current Consensus state. Exploitation of a Safe transaction still requires the Safe owner threshold.
- **A target-chain guard has not received the latest rollover certificate — Verdict: Mitigated** — attestations under the unknown new key revert until the certificate chain is relayed. Updates are permissionless and can be delivered before activation, while older known-key attestations and the announcement escape hatch remain available. This is primarily an operational availability dependency.

- **A rollover certificate is replayed or submitted by a malicious relayer — Verdict: Mitigated** — the signature binds the rollover fields and Consensus domain, the parent must already be trusted, and recording an already-known pair is a no-op.

Announcement escape hatch

- **An announced transaction executes before its delay or after expiry — Verdict: Impossible** — consumption succeeds only within the inclusive `[activeFrom, activeUntil]` interval.
- **A caller changes announced transaction parameters at execution — Verdict: Mitigated** — the nonce-free announcement hash covers every `execTransaction` field except the Safe nonce, including calldata and all refund parameters. State is separately scoped by the caller address.
- **Unrelated Safe transactions invalidate an announcement — Verdict: By design** — the nonce is deliberately excluded so an announcement survives unrelated nonce advances and can execute at the Safe's then-current nonce.
- **An attested execution leaves a matching announcement available — Verdict: By design** — attestation and announcement are independent authorization paths. The attestation path takes precedence and intentionally does not consume a matching announcement.
- **A caught target-call failure consumes the announcement — Verdict: By design** — consumption tracks the selected authorization path rather than execution success. With nonzero `safeTxGas` or `gasPrice`, Safe may retain a failed inner execution and the announcement remains consumed; a finalize-or-restore state machine was considered and deferred.
- **An untrusted party submits a matured, fully signed transaction — Verdict: By design** — the announcement alone is insufficient. Once owners release a fully signed transaction, any holder may relay it during the active window.
- **A matured announcement permanently changes Safe security configuration — Verdict: By design** — the escape hatch authorizes the exact announced transaction without restricting its effects. It may remove the guard, enable a module, change owners, install a fallback handler, approve assets, or perform a delegatecall after the delay.

- **Accounts that do not currently use the guard call its stateful entry points — Verdict: Audit finding (I-02)** — `announceTransaction`, `cancelAnnouncement`, and `checkTransaction` do not verify that the caller is a Safe currently configured with this guard. State remains scoped to `msg.sender`, so this is a defense-in-depth and state-integrity issue without identified direct asset impact.
- **The guard is also configured as the Safe fallback handler — Verdict: Audit finding (L-02)** — Safe fallback forwarding makes the guard observe `msg.sender == Safe` for calls initiated by arbitrary external accounts. Because appended caller bytes are accepted as trailing calldata, an attacker can create, cancel, or consume Safe-scoped announcement state without owner authorization.

Safe integration and execution

- **An enabled module bypasses Safenet authorization — Verdict: By design** — Safe module transactions do not invoke transaction-guard hooks. Deployments must prohibit modules or treat every enabled module as a fully trusted bypass.
- **A call target reenters the guard or Safe — Verdict: Mitigated** — authorization is checked before the Safe performs the target call, and attested replay remains bound to the consumed Safe nonce. Any separate reentrant Safe execution must satisfy Safe's own authorization and nonce rules.
- **A delegatecall target changes Safe storage or security configuration — Verdict: By design** — `delegatecall` has full Safe-context authority for that execution and must be explicitly authorized through the attestation or delayed-announcement path.
- **The post-execution hook does not reconcile authorization state — Verdict: By design** — all authorization occurs in `checkTransaction`; `checkAfterExecution` is intentionally empty, and announcement consumption intentionally records path selection rather than success.
- **Owners remove the guard or enable a bypassing module — Verdict: By design** — the guard delays rather than permanently removes the owner quorum's authority. An exact configuration-changing transaction can execute through a Safenet attestation or after a matured announcement.

- **The guard is absent, installed in the wrong slot, or used with an incompatible Safe — Verdict: By design** — protection depends on correct Safe integration. The guard does not authenticate Safe bytecode or independently force the Safe to invoke its hooks.

Deployment, encoding, and observability

- **The guard is deployed with the wrong Consensus domain, genesis key, delay, or window — Verdict: By design** — these parameters are immutable and deployment-trusted. Domain or genesis mistakes require redeployment, while incorrect timing changes the intended liveness and response window.
- **Consensus, validators, and the guard hash different messages — Verdict: Mitigated** — the on-chain components share `ConsensusMessages`, and the guard reconstructs the canonical nonce-bound Safe transaction hash before verifying the FROST signature. Off-chain integrations must use the same EIP-712 schemas and encoding.
- **A normal Safe signature suffix is misread as an attestation — Verdict: Mitigated** — the fixed terminal type hash identifies the versioned signature extension independently of ordinary signature bytes, and the recognized payload length is validated.
- **Wallet software or a relayer does not support the nonstandard signature extension — Verdict: By design** — the fixed 320-byte trailer is an accepted integration requirement; an absent trailer selects the announcement path, while a malformed recognized trailer fails closed.
- **Announcement parameters are absent from `TransactionAnnounced` logs — Verdict: By design** — the event deliberately contains only the Safe, announcement hash, and time window. Full parameters are recoverable from ordinary `announceTransaction` calldata, although dynamically generated nested calls may require traces.
- **No administrator can repair configuration or prune epoch and announcement state — Verdict: By design** — the contract has no privileged administrative role or upgrade function. Recovery occurs through redeployment and a delayed Safe configuration transaction.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	1	1	1
High	-	-	-
Medium	-	-	-
Low	2	2	0
Informational	2	2	0
Total	5	5	1

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
C-01	Auto-allowed calls can drain Safe assets through unattested gas refunds	Critical	Fixed
L-01	Indefinite attestations allow transactions to outlive the policy that approved them	Low	Acknowledged
L-02	Announcement state becomes permissionless when the guard is configured as the Safe's fallback handler	Low	Acknowledged
I-01	Historical FROST keys remain permanently authoritative on target chains	Informational	Acknowledged
I-02	Guard entry points accept calls from accounts that do not use the guard	Informational	Acknowledged

Critical Severity Issues

C-01: Auto-allowed calls can drain Safe assets through unattested gas refunds

Severity: Critical	Impact: High	Likelihood: High
Files: contracts/src/guard/SafenetGuard.sol	Status: Fixed	

Description:

`SafenetGuard` auto-allows Safe calls to `announceTransaction` and `cancelAnnouncement` so that owners can manage the escape hatch without a FROST attestation. This authorization bypass applies to the entire Safe transaction, including its gas-refund fields. A compromised owner quorum can therefore use an otherwise valid announcement-management call to transfer assets out of the Safe without a Safenet attestation and without waiting for an announcement to mature.

`checkTransaction()` returns immediately when `_isAutoAllowed()` succeeds. The helper validates only the call target, ETH value, operation, and calldata selector. It does not validate `baseGas`, `gasPrice`, `gasToken`, or `refundReceiver`.

In addition to the requested inner call, Safe pays a gas refund after execution whenever `gasPrice` is nonzero. In Safe v1.5.0, the refund is sent to `refundReceiver` and is calculated as:

```
ERC-20:    (gasUsed + baseGas) * gasPrice
Native ETH: (gasUsed + baseGas) * min(gasPrice, tx.gasprice)
```

The refund is paid in the selected `gasToken`, or in native ETH when `gasToken` is zero. [Safe.sol#L219](#) performs this payment after the guard hook, and [Safe.sol#L241](#) implements the calculation.

An attacker with the Safe owner threshold can submit an `execTransaction` with the following shape:

- `to` is the `SafenetGuard` and `data` is a valid `announceTransaction(...)` call;

- `value` is zero and `operation` is `CALL`, satisfying `_isAutoAllowed()`;
- `gasToken` is a valuable ERC-20 held by the Safe, or zero for ETH;
- `refundReceiver` is attacker-controlled; and
- `baseGas` and `gasPrice` are chosen to make the refund equal to the desired withdrawal amount.

No FROST trailer is needed. The guard returns through the auto-allow branch, after which Safe transfers the refund. For an ERC-20, an attacker can choose `gasPrice = 1` and set `baseGas` just below the desired token amount. The small `gasUsed` component can be accounted for when sizing the transaction. The same primitive applies to ETH because `baseGas` is not capped.

This is a complete bypass of SafenetGuard's security model. The normal attestation path includes every refund field in the Safe transaction hash signed by Safenet. The matured-announcement path includes the same fields in the delayed announcement hash. The auto-allow path is the only path that authorizes neither the inner transaction nor its refund through Safenet or the delay mechanism.

The impact is loss of an arbitrary amount of any ERC-20 balance, and of native ETH, held by a guarded Safe. The attack requires a compromised owner quorum, but that is the precise threat SafenetGuard is intended to mitigate. It requires no validator compromise, oracle manipulation, relayer cooperation, or timing race, making exploitation straightforward once the owner threshold is compromised.

Recommendations:

Disable gas refunds on the auto-allow path before returning. Announcement management does not need a refund, while the attestation and matured-announcement paths can continue to support legitimate, explicitly authorized refunds.

```
if (_isAutoAllowed(to, value, data, operation)) {
    if (baseGas != 0 || gasPrice != 0 || gasToken != address(0) || refundReceiver
!= address(0)) {
        revert RefundNotAllowedOnAutoAllowedCall();
    }
    return;
}
```

Customer Response:

Fixed in [PR#862](#).

Fix Review:

Fix confirmed.

Low Severity Issues

L-01: Indefinite attestations allow transactions to outlive the policy that approved them

Severity: Low	Impact: Medium	Likelihood: Low
Files: contracts/src/guard/SafenetGuard.sol	Status: Acknowledged	

Description:

SafenetGuard treats a valid FROST attestation as an indefinite authorization for the exact nonce-bound Safe transaction. The signed **TransactionProposal** commits to the epoch, oracle, oracle data, and Safe transaction hash, but contains no issuance time, expiration, or policy version. At execution, the guard checks only that the (**groupKey**, **epoch**) pair was recorded and that the signature is valid. Historical epoch keys remain trusted forever.

The deadlines in **SentinelOracle** bound only the commit-reveal process; they do not limit a completed signature. The Safe nonce prevents replay at another nonce, but does not provide freshness.

This matters because sentinel decisions can change over time. Operators can update blocklists, remote checks, checker logic, or the set of honored oracles, and a previously clean destination can later be identified as malicious. Even if the current sentinel set would deny the same transaction, the previous approval remains executable. The guard does not consult a current result. Attestations cannot be directly revoked and remain valid indefinitely while the signed Safe nonce remains current. Its effective rule is therefore:

```
Executable(transaction) = any historical sentinel policy approved transaction
```

For example, an attacker can use this during a coordinated campaign against multiple compromised Safes:

1. Before executing any drain, permissionlessly request attestations for exact transactions at each Safe's current nonce, sending funds to destinations that are not yet flagged.
2. Collect the attestations and then begin executing the campaign.
3. After the first drains, sentinel operators or threat-intelligence providers classify the destinations as malicious and update their checks;
4. The attacker continues executing the remaining pre-attested transactions, although fresh requests for them would now be denied.

`Consensus.proposeTransaction` accepts a caller-supplied transaction without verifying Safe owner authorization or that its nonce is current. The attacker can therefore stage the campaign without submitting owner signatures or executing anything from the victim Safes. No validator, sentinel, or arbitrator compromise is required.

The impact is execution of transactions the current Safenet network would reject, preventing reactive policy updates from containing the remaining campaign and potentially resulting in loss of assets held by the affected Safes. Exploitation still requires the exact attested transactions and nonces, the Safe owner thresholds, and a policy or threat-state change. An attacker may also be able to submit all drains in parallel before defenders react, which limits the incremental impact in some cases. Nevertheless, compromised owners are the principal threat SafenetGuard is intended to mitigate, and an outstanding attestation can survive later detection until the Safe nonce advances. Owners can advance the nonce through an auto-allowed announcement-management transaction without waiting for an announcement to mature.

Recommendations:

Add a protocol-bounded validity deadline to the FROST-signed transaction message and attestation trailer:

```
TransactionProposalV2(  
    uint64 epoch,  
    address oracle,  
    bytes oracleData,  
    bytes32 safeTxHash,  
    uint64 validUntil  
)
```

The guard should reject an attestation after the signed deadline:

```
if (block.timestamp > validUntil) {  
    revert AttestationExpired();  
}
```

The protocol must set or cap the maximum lifetime so that an untrusted proposer cannot choose an arbitrary far-future deadline. The chosen lifetime should accommodate the oracle and signing windows while bounding how long an old decision remains executable.

Customer Response:

Acknowledged. Accepted risk.

L-02: Announcement state becomes permissionless when the guard is configured as the Safe's fallback handler

Severity: Low	Impact: Low	Likelihood: Low
Files: contracts/src/guard/SafenetGuard.sol	Status: Acknowledged	

Description:

The three operations that modify announcement state rely on `msg.sender` to identify the Safe:

- `announceTransaction()` creates an announcement under `msg.sender`.
- `cancelAnnouncement()` cancels an announcement under `msg.sender`.
- `checkTransaction()` consumes a matured announcement under `msg.sender`.

If `SafenetGuard` is also configured as the Safe's fallback handler, anyone can perform these operations for the Safe. Safe forwards calls to its fallback handler with `msg.sender` equal to the Safe and appends the original external caller's address to the calldata:

```
mstore(add(ptr, calldatasize()), shl(96, caller()))
let success := call(gas(), handler, 0, ptr, add(calldatasize(), 20), 0, 0)
```

Solidity accepts the appended bytes as trailing calldata. An attacker can therefore call the Safe with calldata for any of the three functions, causing the guard to treat the call as originating from the Safe without an owner-authorized `execTransaction`.

As a result, an arbitrary caller can create malicious announcements, cancel legitimate announcements, or consume matured announcements without executing the corresponding transaction.

Recommendations:

Prohibit and document configuring `SafenetGuard` as a fallback handler. Deployment and Safe-setup tooling should verify that the contract is configured only as the transaction guard.

For defense in depth, the announcement-modifying functions should reject calls from a Safe whose fallback-handler storage slot contains `address(this)`.

Customer Response:

Acknowledged. Documented in [PR#872](#).

Informational Severity Issues

I-01: Historical FROST keys remain permanently authoritative on target chains

Files:

contracts/src/guard/SafenetGuard.sol

Description:

SafenetGuard permanently trusts every recorded (**groupKey**, **epoch**) pair. It does not track an active epoch or retire old keys. As a result, compromising a threshold of shares from any historical FROST group restores full Safenet signing authority on every target-chain guard that knows that pair.

The guarantees differ between Gnosis Chain and the target chains. **Consensus** tracks the active epoch and uses the active group when proposing transactions and staging rollovers. By contrast, **SafenetGuard** accepts an attestation whenever its supplied (**groupKey**, **epoch**) pair exists in the epoch forest and the raw FROST signature is valid. It does not require evidence that the signature came from **FROSTCoordinator**, that **Consensus** recorded an attestation, or that an oracle request existed.

An attacker with a threshold of historical shares can therefore generate a valid FROST signature offline. No Coordinator or on-chain signing ceremony is needed on the target chain. The attacker can sign an arbitrary **TransactionProposal** using the historical epoch and attach it to the Safe owner signatures. They can also use the historical key to sign an epoch rollover that introduces an attacker-controlled key, because **updateEpoch** allows any known historical pair to authorize a later epoch.

The attacker still needs the Safe owner threshold to execute a Safe transaction. However, Safenet is intended to provide additional protection when Safe owners are compromised. If both thresholds are compromised, the historical FROST key bypasses the oracle and current validator set, allowing arbitrary owner-authorized transactions to pass the guard. Rotation cannot contain the compromise because the historical key is never revoked.

Exploitation requires compromising a threshold of historical secret shares, so it is less likely than an ordinary permissionless attack. The design assumes that validators securely erase old shares after outstanding signing sessions finish. This assumption may fail if shares survive in backups,

snapshots, retired validator machines, or incident-response images. The exposure also grows over time because every epoch adds another credential that must remain unrecoverable forever. The behavior is documented and intentional, so this is classified as informational, but its conditional impact is significant.

Recommendations:

Replace permanent forest-wide authorization with a bounded epoch lifecycle. Target-chain guards should accept the exact active key and, for in-flight transactions, the previous key for a short grace period. Only the active key should be able to authorize the next rollover. After the grace period, the previous key should become unusable for both attestations and rollovers.

Authenticate activation using a finalized Gnosis state proof, a bridge, or a second FROST-signed activation certificate produced after the rollover becomes active. A shared epoch registry per target chain can reduce the operational burden to one permissionless and idempotent update per epoch.

Customer Response:

Acknowledged. Accepted risk.

I-02: Guard entry points accept calls from accounts that do not use the guard

Description:

`announceTransaction`, `cancelAnnouncement`, `checkTransaction`, and `checkAfterExecution` do not verify that the caller is a Safe currently configured to use this guard.

These functions only have meaningful protocol semantics when called in the context of a Safe protected by `SafenetGuard`. Accepting calls from other accounts may create irrelevant announcement state or trigger guard logic outside its intended lifecycle. No direct security impact has been identified because this state remains scoped to `msg.sender`, and a caller that already controls a Safe through an unrestricted path does not gain additional authority from pre-arming an announcement.

This is therefore a defense-in-depth and state-integrity improvement rather than a security vulnerability.

Recommendations:

Require `msg.sender` to be a Safe currently configured with `address(this)` as its transaction guard before modifying Safe-scoped announcement state. Apply this check to `announceTransaction`, `cancelAnnouncement`, and the announcement-consumption branch of `checkTransaction`.

Do not apply the current-guard check to `checkAfterExecution`. Safe calls the guard that was active before execution, even when the transaction successfully replaces or removes it. Since `checkAfterExecution` is currently pure and has no effects, unrestricted direct calls are harmless.

Customer Response:

Acknowledged.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.